AlphaStar Unplugged: Large-Scale Offline Reinforcement Learning

Michaël Mathieu[*,1], **Sherjil Ozair**[*,1], **Srivatsan Srinivasan**[*,1], **Caglar Gulcehre**[*,1], **Shangtong Zhang**[*,2], **Ray Jiang**[*,1], **Tom Le Paine**[*,1], **Richard Powell**[1], **Konrad Żołna**[1], **Julian Schrittwieser**[1], **David Choi**[1], **Petko Georgiev**[1], **Daniel Toyama**[1], **Aja Huang**[1], **Roman Ring**[1], **Igor Babuschkin**[1], **Timo Ewalds**[1], **Mahyar Bordbar**[1], **Sarah Henderson**[1], **Sergio Gómez Colmenarejo**[1], **Aäron van den Oord**[1], **Wojciech Marian Czarnecki**[1], **Nando de Freitas**[1] **and Oriol Vinyals**[1]
[*]Equal contributions, [1]Google DeepMind, [2]University of Virgina

arXiv:2308.03526v1 [cs.LG] 7 Aug 2023

StarCraft II is one of the most challenging simulated reinforcement learning environments; it is partially observable, stochastic, multi-agent, and mastering StarCraft II requires strategic planning over long time horizons with real-time low-level execution. It also has an active professional competitive scene. StarCraft II is uniquely suited for advancing offline RL algorithms, both because of its challenging nature and because Blizzard has released a massive dataset of millions of StarCraft II games played by human players. This paper leverages that and establishes a benchmark, called *AlphaStar Unplugged*, introducing unprecedented challenges for offline reinforcement learning. We define a dataset (a subset of Blizzard's release), tools standardizing an API for machine learning methods, and an evaluation protocol. We also present baseline agents, including behavior cloning, offline variants of actor-critic and MuZero. We improve the state of the art of agents using only offline data, and we achieve 90% win rate against previously published AlphaStar behavior cloning agent.

Keywords: Starcraft II, Offline RL, Large-scale learning

1. Introduction

Deep Reinforcement Learning is dominated by online Reinforcement Learning (RL) algorithms, where agents must interact with the environment to explore and learn. The online RL paradigm achieved considerable success on Atari (Mnih et al., 2015), Go (Silver et al., 2017), StarCraft II (Vinyals et al., 2019), DOTA 2 (Berner et al., 2019), and robotics (Andrychowicz et al., 2020). However, the requirements of extensive interaction and exploration make these algorithms unsuitable and unsafe for many real-world applications. In contrast, in the offline setting (Fu et al., 2020; Fujimoto et al., 2019; Gulcehre et al., 2020), agents learn from a fixed dataset previously logged by humans or other agents. While the offline setting would enable RL in real-world applications, most offline RL benchmarks such as D4RL (Fu et al., 2020) and RL Unplugged (Gulcehre et al., 2020) have mostly focused on simple environments with data produced by RL agents. More challenging benchmarks are needed to make progress towards more ambitious real-world applications.

To rise to this challenge, we introduce *AlphaStar Unplugged*, an offline RL benchmark, which uses a dataset derived from replays of millions of humans playing the multi-player competitive game of StarCraft II. StarCraft II continues to be one of the most complex simulated environments, with partial observability, stochasticity, large action and observation spaces, delayed rewards, and multi-agent dynamics. Additionally, mastering the game requires strategic planning over long time horizons, and real-time low-level execution. Given these difficulties, breakthroughs in AlphaStar Unplugged will likely translate to many other offline RL settings, potentially transforming the field.

Additionally, unlike most RL domains, StarCraft II has an independent leaderboard of competitive

Corresponding author(s): mmathieu@google.com
© 2023 DeepMind. All rights reserved

human players over a wide range of skills. It constitutes a rich and abundant source of data to train and evaluate offline RL agents.

With this paper, we release the most challenging large-scale offline RL benchmark to date, including the code of canonical agents and data processing software. We note that removing the environment interactions from the training loop significantly lowers the compute demands of StarCraft II, making this environment accessible to far more researchers in the AI community.

Our experiments on this benchmark suggest that families of algorithms that are state-of-the-art on small scale benchmarks do not perform well here, e.g. Return Conditioned Behavior Cloning (Emmons et al., 2021; Srivastava et al., 2019), Q-function based approaches (Fujimoto et al., 2019; Gulcehre et al., 2021; Wang et al., 2020), and algorithms that perform off-policy evaluation during learning (Schrittwieser et al., 2021b). These approaches sometimes fail to win a single game against our weakest opponent, and all fail to outperform our unconditional behavior cloning baseline.

However, it has also provided insights on how to design successful agents. So far all of our successful approaches are so-call one-step offline RL approaches (Brandfonbrener et al., 2021; Gulcehre et al., 2021). Generally, our best performing agents follow a two-step recipe: first train a model to estimate the behavior policy and behavior value function. Then, use the behavior value function to improve the policy, either while training or during inference. We believe sharing these insights will be valuable to anyone interested in offline RL, especially at large scale.

2. StarCraft II for Offline Reinforcement Learning

StarCraft is a real-time strategy game in which players compete to control a shared map by gathering resources and building units and structures. The game has several modes, such as team games or custom maps. For instance, the StarCraft Multi-Agent Challenge (Samvelyan et al., 2019) is an increasingly popular benchmark for Multi-Agent Reinforcement Learning and includes a collection of specific tasks.

In this paper, we consider StarCraft II as a two-player game, which is the primary setting for StarCraft II. This mode is played at all levels, from casual online games to professional esport. It combines high-level reasoning over long horizons with fast-paced unit management. There are numerous strategies for StarCraft II with challenging properties presenting cycles and non-transitivity, especially since players start the game by selecting one of three alien *races*, each having fundamentally different mechanics, strengths and weaknesses. Each game is played on one of the several *maps*, which have different terrain and can affect strategies.

StarCraft II has many properties making it a great environment to develop and benchmark offline reinforcement learning algorithms. It has been played online for many years, and millions of the games were recorded as *replays*, which can be used to train agents. On the other hand, evaluation of the agents can be done by playing against humans — including professional players — the built-in bots, scripted bots from the community, or even the stronger online RL agents such as AlphaStar (Vinyals et al., 2019) or TStarBot (Han et al., 2021). Finally, we highlight a few properties of StarCraft II that make it particularly challenging from an offline RL perspective.

Action space. When learning from offline data, the performance of algorithms depends greatly on the availability of different state-action pairs in the data. We call this *coverage* — the more state-action pairs are absent, *i.e.* the lower the coverage, the more challenging the problem is. StarCraft II has a highly structured action space. The agent must select an action type, select a subset of its units to apply the action to, select a target for the action (either a map location or a visible unit), and decide when to observe and act next. In our API, we can consider there are approximately 10^{26} possible

actions per game step. In comparison, Atari has only 18 possible actions per step. This makes it almost impossible to attain high state-action coverage for StarCraft II.

Stochastic environment. Stochastic environments may need many more trajectories to obtain high state-action coverage. The game engine has a small amount of stochasticity itself, but the main source of randomness is the unknown opponent policy, which is typically not deterministic. In conrast, in the Atari environment, stochasticity arises only from sticky actions (Machado et al., 2018).

Partial Observability. StarCraft II is an imperfect information game. Players only have information about opponent units that are within the field of view of the player's own units. As a result, players need to scout, *i.e.* send their units around the map to gather information about the current state of the game, and may need it at a later point in the game. On the other hand, a memory of the 3 previous frames is usually considered sufficient for Atari.

Data. For StarCraft II, we have access to a dataset of millions of human replays. These replays display a wide and diverse range of exploration and exploitation strategies. In comparison, the existing benchmarks (Agarwal et al., 2020; Gulcehre et al., 2020) have a bias toward datasets generated by RL agents.

3. AlphaStar Unplugged

We propose AlphaStar Unplugged as a benchmark for offline learning on StarCraft II. This work builds on top of the StarCraft II Learning Environment and associated replay dataset (Vinyals et al., 2017a), and the AlphaStar agents described in Vinyals et al. (2019), by providing a few key components necessary for an offline RL benchmark:

- **Training setup.** We fix a dataset and a set of rules for training in order to have fair comparison between methods.
- **Evaluation metric.** We propose a set of metrics to measure performance of agents.
- **Baseline agents.** We provide a number of well tuned baseline agents.
- **Open source code.** Building an agent that performs well on StarCraft II is a massive engineering endeavor. We provide a well-tuned behavior cloning agent which forms the backbone for all agents presented in this paper[1].

3.1. Dataset

About 20 million StarCraft II games are publicly available through the replay packs[2]. For technical reasons, we restrict the data to StarCraft II versions 4.8.2 to 4.9.2 which leaves nearly 5 million games. They come from the StarCraft II *ladder*, the official matchmaking mechanism. Each player is rated by their *MMR*, a ranking mechanism similar to *Elo* (Elo, 1978). The MMR ranges roughly from 0 to 7000. Figure 1 shows the distribution of MMR among the episodes. In order to get quality training data, we only use games played by players with MMR greater than 3500, which corresponds to the top 22% of players. This leaves us with approximately 1.4

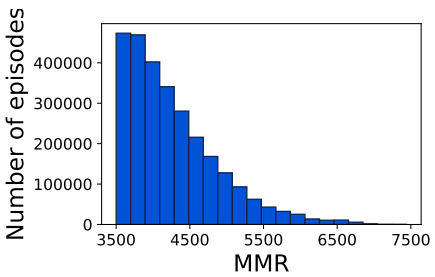


Figure 1 | Histogram of player MMR from replays used for training.

[1] We open-sourced our architecture, data pipeline, dataset generation scripts and supervised learning agent in https://github.com/deepmind/alphastar

[2] https://github.com/Blizzard/s2client-proto/tree/master/samples/replay-api

million games. Each game forms two episodes from a machine learning point of view — one for each side, since we consider two-player games — so there are 2.8 million episodes in the dataset. This represents a total of more than 30 years of game played. These replays span over two different balance patches, introducing some subtle differences in the rules of StarCraft II between the older and the more recent games, which are small enough to be ignored[3]. In addition, the map pool changed once during this period, so the games are played on a total of 10 different maps[4].

The average two-player game is about 11 minutes long which corresponds to approximately 15,000 internal game steps in total. This poses a significant modeling challenge, making training harder and slower. Therefore, we shorten trajectories by only observing the steps when the player took an action. We augment each observation by adding the *delay* which contains the number of internal game steps until the next action, and we discard the internal steps in-between. This cuts the effective length of the episode by 12 times, and is similar to what was done in Vinyals et al. (2019).

Each episode also contains metadata, the most important ones being the outcome, which can be 1 for a victory, 0 for a draw[5] and −1 for a defeat, as well as the MMR of each player. The games were played online using Blizzard's matchmaking system which ensures that in the vast majority of games, both players have a similar MMR.

The replays are provided by Blizzard and hosted on their servers. The data is anonymized, and does not contain personal information about the players. The full dataset represents over 30 years of game play time, in the form of 21 billion internal game steps. This corresponds to 3.5 billion training observations.

3.2. Training restrictions

During training, we do not allow algorithms to use data beyond the dataset described in Section 3.1. In particular, the environment cannot be used to collect more data. However, online policy evaluation is authorized, *i.e.* policies can be run in the environment to measure their performance. This may be useful for hyperparameter tuning.

Unlike the original AlphaStar agents, agents are trained to play all three races of StarCraft II. This is more challenging, as agents are typically better when they are trained on a single race. They are also trained to play on all 10 maps available in the dataset.

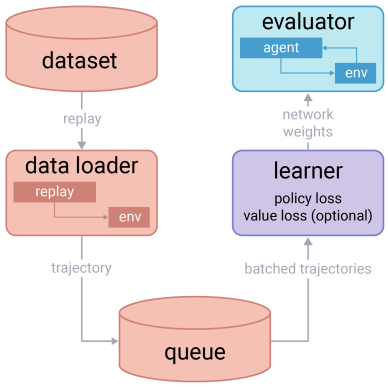


Figure 2 | Training procedure.

In our experiments, we tried to use the same number of training inputs whenever possible — of the order of $k_{max} = 10^{10}$ observations in total — to make results easier to compare. However this should be used as a guideline and not as a hard constraint. The final performance reached after each method eventually saturates is a meaningful comparison metric, assuming each method was given enough compute budget.

[3]However, the version of the game is available for each episode, so one could decide to condition the agent on the version.

[4]Acropolis, Automaton, Cyber Forest, Kairos Junction, King's Cove, New Repugnancy, Port Aleksander, Thunderbird, Turbo Cruise '84, Year Zero.

[5]Draws are rare in StarCraft II, but can happen if no player can fulfill the win-condition.

3.3. Evaluation protocol

Numerous metrics can be used to evaluate the agents. On one hand, the easiest to compute — and least informative — is simply to look at the value of the loss function. On the other hand, perhaps the most informative — and most difficult to compute — metric would be to evaluate the agent against a wide panel of human players, including professional players. In this paper, we propose a compromise between these two extremes. We evaluate our agents by playing repeated games against a fixed selection of 7 opponents: the `very_hard` built-in bot[6], as well as a set of 6 reference agents presented below.

During training, we only evaluate the agents against the `very_hard` bot, since it is significantly less expensive, and we mostly use that as a validation metric, to tune hyper-parameters and discard non-promising experiments.

Fully trained agents are evaluated against the full set of opponents presented above, on all maps. We combine these win rates into two aggregated metrics while uniformly sampling the races of any pair of these agents: *Elo rating* (Elo, 1978), and *robustness*. Robustness is computed as one minus the minimum win rate over the set of reference agents. See details of the metrics computation in Appendix A.2.

4. Reference Agents

As explained in Section 3, we provide a set of 6 reference agents, which can be used both as baselines and for evaluation metrics. In this section, we detail the methodology and algorithms used to train them. The implementation details can be found in Appendix A.3, and results in Section 5.11.

4.1. Definitions

The underlying system dynamics of StarCraft II can be described by a *Markov Decision Process*[7] *(MDP)* (Bellman, 1957). An MDP, $(\mathcal{S}, \mathcal{A}, P, r, \mathcal{I})$, consists of finite sets of states $\mathcal{S}$ and actions $\mathcal{A}$, a transition distribution $P(s'|s,a)$ for all $(s,a,s') \in \mathcal{S} \times \mathcal{A} \times \mathcal{S}$, a reward function[8] $r : \mathcal{S} \to \mathbb{R}$, and an initial state distribution $\mathcal{I} : \mathcal{S} \to [0,1]$. In the offline setting, the agent does not interact with the MDP but learns only from a dataset $\mathcal{D}$ containing *episodes*, made of sequences of state and actions (s_t, a_t). We denote $\mathbf{s}$ the sequence of all states in the episode, and $len(\mathbf{s})$ its length. A *policy* is a probability distribution over the actions given a state. The dataset $\mathcal{D}$ is assumed to have been generated by following an unknown *behavior policy* μ, such that $a_t \sim \mu(\cdot|s_t)$ for all $t < len(\mathbf{s})$.

As explained in Section 3.1, observed states are a subset of the internal game steps. We call *delay* the number of internal game steps between two observed internal game steps, which corresponds to the amount of real time elapsed between the two observations[9]. Given states and action (s_t, a_t, s_{t+1}), we note $d(a_t)$ the delay between states s_t and s_{t+1}. Note that the delay at step t is referring to the step $t+1$, not $t-1$. This is needed for inference, since the environment must be provided with the number of internal steps to skip until the next observation. Therefore the delay must be part of the action.

Given a policy π and a state s_t, we define the expected discounted return $v^\pi(s_t)$ as the expected sum of the discounted rewards obtained if we follow π from s_t. The discount between two steps is

[6] The `very_hard` bot is not the strongest built-in bot in StarCraft II, but it is the strongest whose strength does not come from unfair advantages which break the game rules.

[7] Strictly speaking, we have a Partially Observable MDP, but we simplify this for ease of presentation.

[8] In the usual definition of an MDP, the reward is a function of the state and the action. But in StarCraft II, the reward is 1 in a winning state, -1 in a losing state, and zero otherwise. So it does not depend on the action.

[9] One internal game step occurs every 45ms.

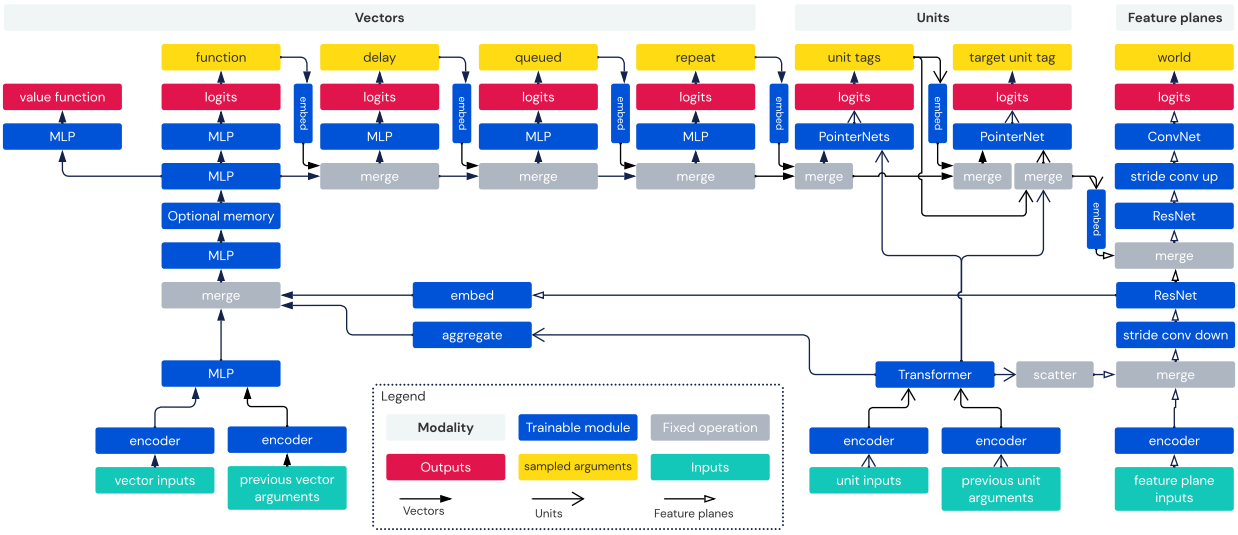


Figure 3 | Illustration of the architecture that we used for our reference agents. Different types of data are denoted by different types of arrows (vectors, units or feature planes).

based on the delay between the steps. In the case of StarCraft II, the reward is the win-loss signal, so it can only be non-zero on the last step of the episode. Therefore we can write

$$v^{\pi}(s_t) = \mathbb{E}_{s_{k+1}\sim(\cdot|s_k,a_k),a_k\sim\pi(\cdot|s_k),\forall k\geq t}\left[\gamma^{D_t(\mathbf{s})} r(\mathbf{s})\right] \qquad \text{with} \qquad D_t(\mathbf{s}) = \sum_{k=t}^{len(\mathbf{s})-1} d(a_k), \tag{1}$$

where $r(\mathbf{s})$ is the reward on the last step of the episode. $D_t(\mathbf{s})$ is simply the remaining number of internal game steps until the end of the episode.

The goal of offline RL is to find a policy π^* which maximizes $\mathbb{E}_{s_0\in\mathcal{I}}[v^{\pi^*}(s_0)]$. Dunring training, we refer to the policy π trained to estimate π^* as the *target policy*.

We use V^{μ} and V^{π} to denote the value functions for the behavior and target policies μ and π, which are trained to estimate v^{μ} and v^{π}, respectively.

We typically train the agent on *rollouts*, *i.e.* sequences of up to K consecutive timesteps, assembled in a minibatch of M independent rollouts. Unless specified otherwise, the minibatches are independent from each other, such that two consecutive minibatches are not correlated.

4.2. Architecture

All our experiments are based on the same agent architecture. It is an improved version of the model used in Vinyals et al. (2019). The full architecture is summarized on Figure 3.

Inputs of the raw StarCraft II API are structured around three modalities: *vectors*, *units* — a list of features for each unit present in the game — and *feature planes* (see Appendix A.1 for more details).

Actions are comprised of seven *arguments*, and can be organized in similar modalities: `function`, `delay`, `queued` and `repeat` as vectors, since each argument is sampled from a single vector of logits. `Unit_tags` and `target_unit_tag` refer to indices in the units inputs. Finally, the `world` action is a 2d point on the feature planes.

We structure the architecture around these modalities:

- Each of the three modalities of inputs is encoded and processed independently using a fitting architecture: MLP for the vector inputs, transformer (Vaswani et al., 2017) for the units input and residual convolutional network (He et al., 2015) for the feature planes. Some of these convolutions are strided so that most of the computation is done at a lower resolution. Arguments of the previous action are embedded as well, with the execption of the `world` previous argument, since we found this causes too much overfitting.
- We use special operations to add interactions between these modalities: we *scatter* units into feature planes, *i.e.* we place the embedding of each unit in its corresponding spatial location on the feature plane. We use a averaging operation to embed the units into the embedded vectors. Feature planes are embedded into vectors using strided convolutions and reshaping, and the reverse operations to embed vectors into feature planes.
- We tried using memory in the vector modality, which can be LSTM (Hochreiter & Schmidhuber, 1997) or Transformer XL (Dai et al., 2019). Most of our results do not use memory (see Section 5.6).
- For the experiments using a value function, we add an MLP on top of the vector features to produce a estimate of the value function.
- Finally, we sample actions. The seven arguments are sampled in the following order: `function`, `delay`, `queued`, `repeat`, `unit_tags`, `target_unit_tag` and `world`. They are sampled autoregressively,[10] *i.e.* each sampled argument is embedded to sample the next one. The first four arguments are sampled from the vector modality. The next two are sampled from the vector and units modalities using pointer networks (Vinyals et al., 2017b), and finally the `world` argument is sampled from the upsampled feature planes. Note that `unit_tags` is actually obtained by sampling the pointer network 64 times autoregressively, so conceptually, `unit_tags` represent 64 arguments.

The exact hyperparameters and details of the architecture can be found in the open-sourced code which can be accessed via `https://github.com/deepmind/alphastar`.

MMR conditioning. At training time, the MMR of the player who generated the trajectory is passed as a vector input. During inference, we can control the quality of the game played by the agent by changing the MMR input. In practice, we set the MMR to the highest value to ensure the agent plays its best. This is similar to Return-Conditioned Behavior Cloning (Srivastava et al., 2019) with the MMR as the reward.

MuZero latent model. For the MuZero experiments, detailed in Section 4.5, we define the latent space $\mathcal{L}$ as the space of vectors before the `function` MLP. We split the model presented above into an encoder $E : \mathcal{S} \to \mathcal{L}$ and two decoder: D_π maps latent states to distributions over actions, and a value function decoder $D_{V^\pi} : \mathcal{L} \to \mathbb{R}$, such that $\pi(\cdot|s) = D_\pi(E(s))$ and $V^\pi(s) = D_{V^\pi}(E(s))$. Note that in our implementation, the decoder D_π actually produces distributions for the `function` and `delay` only. The other arguments are obtained from the estimated behavior policy $\hat{\mu}$. Finally, we add a latent model $L : \mathcal{L} \times \mathcal{A} \to \mathcal{L}$. Given a rollout $((s_0, a_0), \ldots (s_K, a_K))$, we compute:

$$h_0 = E(s_0) \qquad h_{k+1} = L(h_k, a_k) \qquad \pi(\cdot|s_k) = D_\pi(h_k) \qquad V^\pi(s_k) = D_{V^\pi}(h_k) \tag{2}$$

for all $k < K$. Note that s_0 is only the first state of the rollout, but not necessarily the first state of an episode. See Figure 4 for an illustration.

[10] With the exception of `target_unit_tag` and `world`, because no action in the API uses a `target_unit_tag` and a `world` argument at the same time.

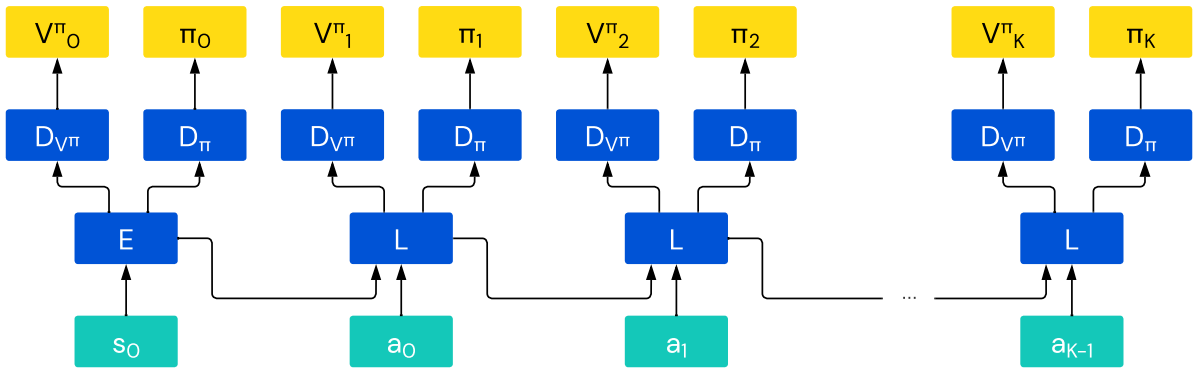


Figure 4 | Illustration of the architecture used for MuZero. E is the encoder, L is the latent model, and D_π and D_{V^π} are the policy and value function decoders, respectively.

4.3. Behavior cloning

Behavior Cloning (BC) agent. Our first reference agent is trained using *behavior cloning*, the process of estimating the behavior policy μ. We learned an estimate $\hat{\mu}$ by minimizing the negative log-likelihood of the action a_t under the policy $\hat{\mu}(\cdot|s_t)$. Given a rollout $\mathbf{s}$, we write

$$L^{BC}(\mathbf{s}) = -\sum_{t=0}^{len(\mathbf{s})-1} \log\left(\hat{\mu}(a_t|s_t)\right). \tag{3}$$

This is similar to training a language model. The procedure is detailed in Algorithm 1 in the Appendix. It is the same procedure that was used by the AlphaStar Supervised agent in Vinyals et al. (2019). In practice, since each action is comprised of seven arguments, there is one loss per argument.
In order to avoid overfitting during behavior cloning, we also used a weight decay loss which is defined as the sum of the square of the network parameters.

Fine-tuned Behavior Cloning (FT-BC) agent. Behavior Cloning mimics the training data, so higher quality data should lead to better performance. Unfortunately, since filtering the data also decreases the number of episodes, generalization is affected (see Section 5.5). In order to get the best of both worlds, we used a method called *fine tuning*. It is a secondary training phase after running behavior cloning on the whole dataset. In this phase, we reduced the learning rate and filter the data to top-tier games. This generalizes better than training only on either set of data, and was already used in Vinyals et al. (2019).

4.4. Offline Actor-Critic

Actor-critic (Barto et al., 1983; Witten, 1977) algorithms learn a target policy π and the value function V^π. In off-policy settings, where the target policy π differs from the behavior policy μ, we compute *importance sampling* ratios $\rho_t(a_t|s_t) = \pi(a_t|s_t)/\mu(a_t|s_t)$ where (s_t, a_t) come from the data, *i.e.* follow the behavior policy. There are many variants of the loss in the literature. The simplest version is called 1-Step Temporal Differences, or TD(0), defined as:

$$L^{TD(0)}(\mathbf{s}) = -\sum_{t=0}^{len(\mathbf{s})-2} \ominus[\rho_t(a_t|s_t)\left(\gamma V^\pi(s_{t+1}) - V^\pi(s_t) + r(s_{t+1})\right)] \log(\pi(a_t|s_t)) \tag{4}$$

where $\ominus$ symbol corresponds to the stop-gradient operation. In this equation, V^π is called the *critic*. The loss can be modified to use N-Step Temporal Differences (Sutton & Barto, 2018) by adding more terms to Equation 4, and it can be further improved by using V-Trace (Espeholt et al., 2018) in order to reduce variance. Note that for simplicity of implementation, we only applied this loss for some of the arguments, namely `function` and `delay`, and we use the behavior policy for the other ones.

We learned the estimated behavior value V^μ by minimizing the *Mean-Squared Error (MSE)* loss:

$$L^{MSE}(\mathbf{s}) = \frac{1}{2} \sum_{t=0}^{len(\mathbf{s})-1} ||V^\mu(s_t) - r(\mathbf{s})||_2^2 \,. \tag{5}$$

Offline Actor-Critic (OAC) agent. Although actor-critic has an off-policy correction term, it was not enough to make it work without adjustments to the pure offline setting.

The behavior policy μ appears in the denominator of ρ, but we do not have access to the behavior policy used by the players, we can only observe their actions. Fortunately, the behavior Cloning agent learns an estimate $\hat{\mu}$ which we used to compute the estimated $\hat{\rho} = \pi/\hat{\mu}$.

The Behavior Cloning policy $\hat{\mu}$ can be used as the starting point for π (*i.e.* used to initialize the weights). This way, the estimated importance sampling $\hat{\rho}$ equals 1 at the beginning of training.

Equation 4 uses V^π as the critic, which is standard with actor-critic methods. This can be done even in offline settings, by using a Temporal Differences loss for the value function (Espeholt et al., 2018). Unfortunately, this can lead to divergence during offline training, which is a known problem (van Hasselt et al., 2018). One solution could be early stopping: the policy π improves at first before deteriorating, therefore we could stop training early and obtain an improved policy π. However, this method requires running the environment to detect when to stop, which is contrary to the rules of AlphaStar Unplugged. Instead, we used V^μ as a critic, and keep it fixed, instead of V^π.

Emphatic Offline Actor-Critic (E-OAC) agent. *N-step Emphatic Traces (NETD)* (Jiang et al., 2021) avoids divergence in off-policy learning under some conditions, by weighting the updates beyond the importance sampling ratios. We refer to Jiang et al. (2021) for details about the computation of the emphatic traces.

4.5. MuZero

MuZero Unplugged (Schrittwieser et al., 2021a) adapts *Monte Carlo Tree Search (MCTS)* to the offline setting. It has been successful on a variety of benchmarks (Dulac-Arnold et al., 2019; Gulcehre et al., 2020). In order to handle the large action space of StarCraft II, we sample multiple actions from the policy and restrict the search to these actions only, as introduced in Hubert et al. (2021). This allows us to scale to the large action space of StarCraft II. We used the latent model presented in Section 4.2, and similarly to the offline actor-critic agents, we only improved the `function` and `delay` from the behavior cloning policy.

MuZero Supervised (MZS) agent. Similarly to the Offline Actor-Critic case, training the target policy π and estimated value function V^π jointly can diverge. In an analog approach, a workaround is to only train the policy to estimate the behavior policy, and use the value and latent model to run MCTS at inference time only. This results in only using the losses for the policy and the value function

Table 1 | Behavior cloning performance with different minibatch sizes M and rollout lengths K.

Minibatch size M	Rollout length K	$M \times K$	win rate vs. `very_hard`
8,192	1	8,192	70%
16,384	1	16,384	79%
256	64	16,384	79%
32,768	1	32,768	84%

for MuZero. In other words, the loss is simply the following loss:

$$L^{MuZero}(s) = L^{BC}(s) + L^{MSE}(s) \tag{6}$$

where the policy and value function are computed using the latent model for all steps except the first one, as shown on Equation 2. Although the loss is similar to standard behavior cloning, using this method can lead to improved performance thanks to the regularization effects of the value function training and the latent model.

MuZero Supervised with MCTS at inference time (MZS-MCTS) agent. The MuZero Unplugged algorithm uses MCTS at training time and inference time. As explained above, policy improvement at training time can lead to divergence. Using MCTS at inference time, on the other hand, is stable and leads to better policies. We use the approach detailed in Hubert et al. (2021) for the inference.

5. Experiments

In this section, we measure the influence of several parameters. For simplicity, we use the win rate against the `very_hard` bot as the metric for these experiments. Most experiments are run in the behavior cloning setting. Due to the cost of running such experiments, we could only train a single model per set of parameters, but the consistency of the conclusions leads us to believe that the results are significant.

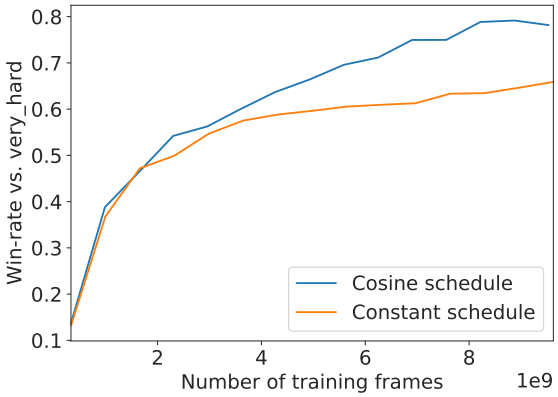
Moreover, Section 5.11 presents the performance of the reference agents on all AlphaStar Unplugged metrics, as well as against the original AlphaStar agents from Vinyals et al. (2019).

In this section, we call number of learner *steps* the number of updates of the weights on minibatches of rollouts of size $M \times K$. We call number of learner *frames* the total number of observations used by the learner, *i.e.* the number of steps multiplied by $M \times K$.

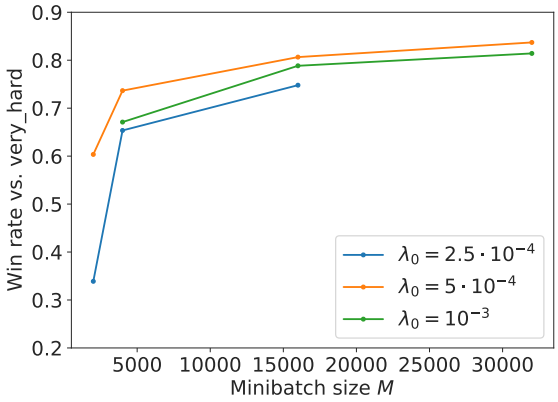
5.1. Minibatch and rollout sizes

The minibatch size M and rollout size K influence the final performance of the models. Table 1 compares some settings in the case of behavior cloning. In all these experiments, the total number of training frames is 10^{10}. We found that more data per step — *i.e.* larger $M \times K$ — leads to better final performance.

There are unfortunately a few constraints to respect. $M \times K$ cannot be increased indefinitely because of the memory usage. The largest value we could use was $16,384$ or $32,768$, depending on the method. Besides, the Offline Actor-Critic and MuZero methods require $K > 1$, and larger values of K stabilize the training.



(a) Comparison of learning rate schedules. The constant learning rate, as well as λ_0, are both set to $5 \cdot 10^{-4}$.



(b) Final performance for different initial learning rates λ_0 and minibatch sizes M.

Figure 5 | Win rate against the `very_hard` bot for different learning rate schedules, on behavior cloning.

5.2. Learning rate

The learning rate λ has a significant influence on the final performance of the agents. We used a cosine learning rate schedule (Loshchilov & Hutter, 2016), parameterized by the initial learning rate λ_0. Some experiments use a ramp-in period over $N_{\text{ramp-in}}$ frames. At frame k, the learning rate is given by

$$\lambda(k) = \min\left(1, \frac{k}{N_{\text{ramp-in}}}\right) \cdot \left(\frac{\lambda_0}{2} \cdot \cos\left(\pi \cdot \frac{k}{k_{max}}\right) + 0.5\right) \tag{7}$$

where k_{max} is the total number of training frames. We compared this schedule to a constant learning rate on Figure 5a.

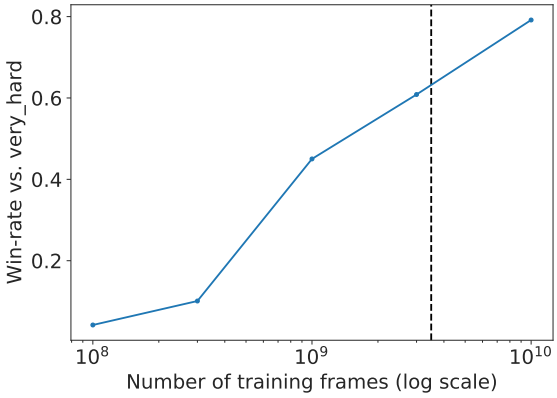
Figure 5b shows the final performance for different values for λ_0 and different minibatch sizes M. Since these experiments are slow, it is common to look at the win rate before the experiment is over and decide to compare the results before convergence. It is noteworthy to mention that it should be avoided to find the optimal λ_0. Indeed, we observed that after only 10^9 steps, the best performance is obtained with the $\lambda_0 = 10^{-3}$, but after the full training, it changes.

In the following experiments, we used $\lambda_0 = 5 \cdot 10^{-4}$ unless specified otherwise. The learning rate schedules used to train the reference agents are detailed in Appendix A.3.

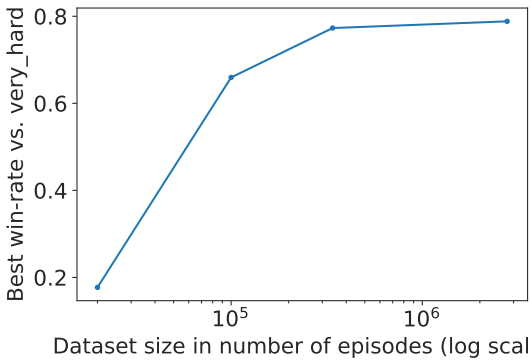
5.3. Number of training frames

As mentioned in Section 3.2, we trained most of our agents over $k_{max} = 10^{10}$ input frames. We measured the behavior cloning performance of the agents trained on fewer frames, as shown on Figure 6a. The performance increases logarithmically with the number of training frames.

Note that the fine-tuned (BC-FT) and offline actor-critic (OAC and E-OAC) reference agents were trained on 10^9 frames, restarting from the behavior cloning agent. Therefore, they were trained on a total on a total of 11 billion frames, whereas the BC, MZS and MZS-MCTS were only trained on 10 billion frames.



(a) Performance when varying the number of training input frames.



(b) Performance when training on 10^{10} frames, with different number of unique episodes in the dataset. Small sizes lead to overfitting so we show the peak win rate over the course of training, instead of the final value.

Figure 6 | Win rate against the `very_hard` bot when scaling the data.

Table 2 | Performance of behavior cloning when using different MMR filtering schemes. Higher quality data also means fewer episodes, therefore worse performance. High quality data for fine-tuning gives the best results.

Main training			Fine-tuning			
MMR	filter	#episodes	MMR	filter	#episodes	win rate vs. `very_hard`
>3500	win+loss	2,776,466				84%
>6000	win+loss	64,894				65%
>6000	win	32,447				51%
>3500	win+loss	2,776,466	>6200	win	21,836	**89%**

5.4. Dataset size

Figure 6b shows the behavior cloning performance for different dataset sizes, *i.e.* number of unique episodes used for training. For all the points on this curve, we trained the model on the full $k_{max} = 10^{10}$ frames, which means that episodes are repeated more often with smaller sized datasets. Unlike most experiments, here we used minibatch size $M = 16,384$ and a learning rate of 10^{-3}.

It is noteworthy that the win rate with only 10% of the episodes in the dataset is close to the best one. This can be used to save storage at very little cost, if it is a concern. However, further reducing the dataset size significantly alters the performance.

5.5. Data filtering

Filtering the data has a large influence on the final performance of the models. Table 2 shows that, for behavior cloning, restricting the training set to fewer, higher quality episodes results in poorer performance. However, training using the full dataset followed by a fine-tuning phase on high quality data works best (BC-FT reference agent).

Table 3 | Comparison of behavior cloning performance against the `very_hard` built-in bot with different implementations of memory.

Memory	Win rate vs. `very_hard`
LSTM	70%
No Memory	84%
Transformer, $k_{max} = 10^{10}$ frames	85%
Transformer, $k_{max} = 2 \cdot 10^{10}$ frames	89%

5.6. Memory

The AlphaStar agent of Vinyals et al. (2019) uses an LSTM module to implement memory. We have tried using LSTM, Transformers and no memory. Surprisingly, we found that no memory performs better than LSTM for behavior cloning, although the final values of the losses are higher.

Results with transformers are more ambivalent. The transformer agent performs similarly to the memory-less agent on 10^{10} training frames. However, although the performance of the memory-less agent saturates beyond $k_{max} = 10^{10}$ frames, transformers do not, and they outperform the memory-less agent if trained for $2 \cdot 10^{10}$ frames. Table 3 summarizes the performance of these agents versus the `very_hard` bot.

Transformer require extensive hyperparameter tuning and longer training times. Therefore all agents presented in the main experiments are memory-less. Using transformers for other Offline RL baselines may result in more pronounced benefits and is an interesting future research direction.

5.7. Model size

Because of the complexity of the model, many parts could be scaled individually, but this would be prohibitive. We chose our standard model size as the largest model which can fit in memory without significant slowdown. Scaling down the width of the model by half leads to significant decrease of the performance, from 83% to 76% win rate against the `very_hard` bot, however scaling down the depth by half (rounding up) barely changes the win rate (82%). In the setup used for our experiments, the training speed does not significantly increase when decreasing the depth, but potential speed gains could be obtained in the future by using smaller models.

5.8. Temperature and sampling

During inference, we sample from the policy $\pi(\cdot|s_t)$ given a state s_t. In practice, the policy is characterized by a *logits* vector y such that:

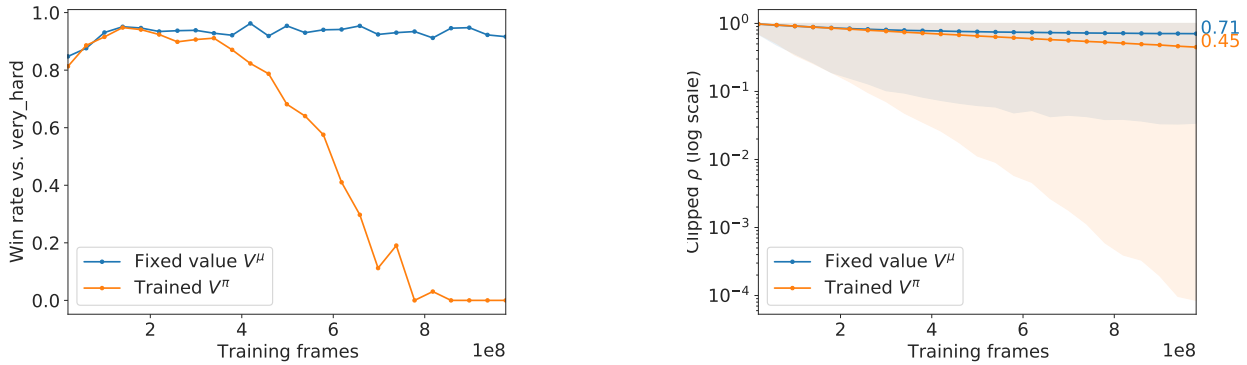
$$\pi(a|s_t) = \frac{\exp(y_a/\beta)}{\sum_{a'=0}^{|\mathcal{A}|} \exp(y_{a'}/\beta)} \tag{8}$$

where β is the *temperature*. During training, the temperature is $\beta = 1$ but it can be changed at inference time, in order to make the policy more peaked.

We found that $\beta = 0.8$ is a good value for the temperature during inference, as shown on Table 4.

Table 4 | Win rate of different agents versus the `very_hard` bot with two different sampling temperatures.

Temperature β during inference	Behavior Cloning	Fine-Tuning	Offline Actor Critic	Emphatic Offline Actor-Critic
1	84%	90%	93%	93%
0.8	88%	95%	98%	97%



(a) Win rate against the `very_hard` bot for offline actor-critic training.

(b) Clipped importance sampling ρ over training.

Figure 7 | Performance and importance sampling values over offline actor-critic training, comparing V^π and V^μ as the critic.

5.9. Critic of offline actor-critic

For the offline actor-critic, we experimented with using the value function of the target policy, V^π, as the critic, instead of using the fixed value function of the behavior policy, V^μ. Figure 7a shows the divergence observed when using V^π. Indeed, although the win rate first increases in both cases, it stays high with V^μ but deteriorates with V^π. On Figure 7b, we can see that the importance sampling ρ (clipped by the V-Trace algorithm) decayed much faster and lower when using V^π. This means that the policy π and μ got further and further apart on the training set and eventually diverged.

5.10. MCTS during training and inference

Our preliminary experiments on using the full MuZero Unplugged algorithm, *i.e.* training with MCTS targets, were not successful. We found that the policy would collapse quickly to a few actions with high (over-)estimated value. While MCTS at inference time improves performance, using MCTS at training time leads to a collapsed policy. To investigate this further, we evaluated the performance of repeated applications of MCTS policy improvement on the behavior policy $\hat{\mu}$ and value V^μ. We do this by training a new MuZero model using MCTS actions of a behavior policy, i.e. $\hat{\nu} = MCTS(\hat{\mu}, V^\mu)$. We found that the MCTS performance of this policy $MCTS(\hat{\nu}, V^\mu)$ is worse than the performance of $\hat{\nu}$ or $MCTS(\hat{\mu}, V^\mu)$. Thus, repeated applications of MCTS do not continue to improve the policy. We believe this is likely due to MCTS policy distribution generating out of distribution action samples with over-estimated value estimates.

Figure 8 compares using MCTS or not during inference. We can see that using MCTS always outperforms not using it, even at the beginning of training.

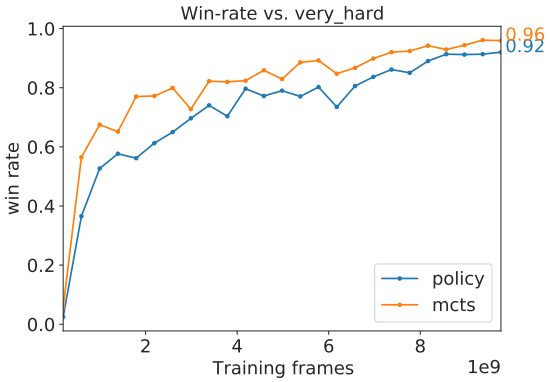


Figure 8 | Comparison of the win rates of the MZS and MZS-MCTS agents over the course of training. Using MCTS outperforms not using it throughout training.

5.11. Evaluation of the reference agents

Table 5 shows the performance of our six reference agents using our three metrics: robustness, Elo and win rate versus the `very_hard` built-in bot. These metrics are explained in Section 3.3. The three best agents utilize offline RL algorithms (highlighted in pale blue).

The full win rate matrix of the reference agents can be seen in Figure 9. A more detailed matrix, split by race, is displayed in Figure 10 in Appendix A.4.

We observe that the MuZero Supervised with MCTS at inference time (MZS) reference agent performs best, although at the cost of slower inference. Generally, we see that the three offline RL methods are ranked closely and significantly higher than behavior cloning. For completeness, we compare with the original AlphaStar agents. The AlphaStar Supervised was trained as three race-specific agents, which is different from the rules of our benchmark (agents should play all races). Therefore, we also compare our agents to a version of AlphaStar Supervised trained to play all races. The win rate of the MZS-MCTS, E-OAC and OAC are 90%, 93% and 90% respectively (see Figure 9). We also note that although offline RL improves upon the behavior cloning baseline, they are far from the online RL performance of AlphaStar Final, which was trained using several orders of magnitude more computing power.

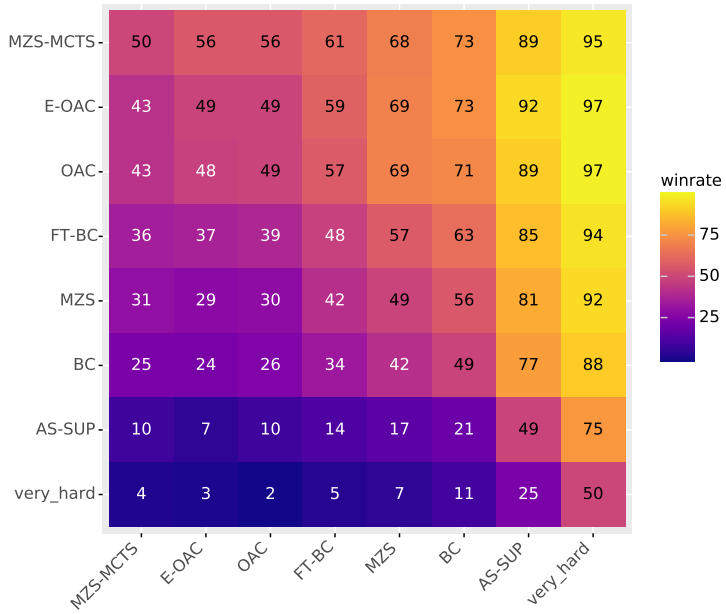


Figure 9 | Win rate matrix of the reference agents, normalized between 0 and 100. Note that because of draws, the win rates do not always sum to 100 across the diagonal. AS-SUP is the original AlphaStar Supervised agent (not race specific).

Table 5 | Evaluation of the 6 reference agents with the proposed metrics. Agents highlighted in pale blue utilize offline RL algorithms, whereas the other 3 rely on behavior cloning. In the bottom portion of this table we show performance of agents from Vinyals et al. (2019). Our BC agent is most comparable to AlphaStar Supervised but performs better due to significant tuning improvements. The other AlphaStar agents highlighted in grey have differences which make their performance not directly comparable to ours.

Agent	Robustness	Elo	vs `very_hard`
MuZero Supervised with MCTS at inference time	50%	1578	95%
Emphatic Offline Actor-Critic	43%	1563	97%
Offline Actor-Critic	43%	1548	98%
Fine-tuned Behavior Cloning	36%	1485	95%
MuZero Supervised	30%	1425	92%
Behavior Cloning	25%	1380	88%
`very_hard` built-in bot	3%	1000	50%
AlphaStar Supervised	8%	1171	75%
AlphaStar Supervised (Race specific networks)	17%	1280	82%
AlphaStar Supervised (Race specific networks + FT)	44%	1545	94%
AlphaStar Final (Race specific networks + FT + Online Learning)	100%	2968	100%

5.12. Additional offline RL baselines

We evaluated several typical off-policy and offline RL baselines such as action-value based methods like deep offline Q-Learning (Agarwal et al., 2020), SARSA (Rummery & Niranjan, 1994), Critic Regularized Regression (CRR) (Wang et al., 2020), Batch-Constrained Deep Q-Learning (BCQ) (Fujimoto et al., 2019), Regularized Behavior Value Estimation (R-BVE) (Gulcehre et al., 2021), Critic-Weighted Policy (CWP) (Wang et al., 2020) and Return Conditioned Behavior Cloning (RCBC) (Srivastava et al., 2019) on AlphaStar Unplugged. We also tried Advantage-Weighted Regression (AWR) (Peng et al., 2019), and Proximal Policy Optimization (PPO) (Schulman et al., 2017). None of those approaches could achieve better results than the agents such as BC and FT-BC. In this section, we will highlight some of those approaches and challenges we faced when we scaled them up to StarCraft II.

Deep offline Q-Learning. We trained offline Q-learning agents based on DQN (Mnih et al., 2015), that are predicting Q-values and policy with the same output layer for only the `function` argument. However, the training of those offline Q-learning agents was very unstable, and they have 0% win rate against the `very_hard` bot. Moreover, typical approaches to improve Q-learning in the such as N-step returns, dueling network architecture (Wang et al., 2016) and double-Q-learning (Hasselt et al., 2016) did not improve the performance of our Q-learning agents. Besides the policies themselves, the accuracy of the action-values predicting the returns was poor.

Offline RL methods using action values. We trained CRR, BCQ, CWP, and R-BVE agents with an action-value Q-head on the `function` argument. CRR and R-BVE achieved very similar results, and neither could provide significant improvements over the BC agent. BVE and R-BVE were very stable in terms of training. For CRR, we also used BVE to learn the Q-values instead of the. On the other hand, CRR, R-BVE, CWP, and BCQ all achieved around 83-84% win rate against the `very_hard` bot.

Return conditioned behavior cloning. (RCBC) We trained a BC agent conditioned on the win-loss return. During inference, we conditioned it on winning returns only, to make it model behavior policy used in winning games. We did not notice any difference, in fact the agent seemed to ignore the return conditioning. We attribute this to the two well-known failure points of RCBC approaches: stochasticity arising due to the noisy opponents, and inability to do trajectory stitching (Brandfonbrener et al., 2022).

6. Discussion

Behavior cloning is the foundation of all agents in this work. The offline RL agents start by estimating the behavior policy using behavior cloning, then improve upon it using the reward signal. This allows them to perform significantly better than the behavior cloning results. Indeed, although the agents are conditioned on the MMR during training, the behavior cloning agents are still fundamentally limited to estimating the behavior policy, ignorant about rewards. As a result, the policy they learn is a smoothed version of all the policies that generated the dataset. In contrast, offline RL methods use rewards to improve learned policies in different ways. Offline Actor-Critic methods use policy-gradient. MCTS at inference time aims at maximizing the estimated return. Even the MuZero Supervised without MCTS and the fine-tuned behavior cloning make use of the reward, and outperform the BC baseline.

We have observed that algorithms originally designed for online learning — even with off-policy corrections — do not work well when applied directly to the full offline RL setting. We attribute this in part to the problem of the *deadly triad* (Sutton & Barto, 2018; Tsitsiklis & Van Roy, 1997; van Hasselt et al., 2018). However, many recent works have found these algorithms can be made more effective simply by making modifications that ensure the target policy stays close to the behavior policy μ, that the value function stays close to V^{μ}, or both. Our results with Actor-Critic and MuZero are in accordance with these findings.

Among all the methods we tried, the reference agents are the ones which led to improved performance. However, we have tried several other methods without success, listed in Section 5.12. We may have failed to find the modifications which would have made these methods perform well on this dataset. However, AlphaStar Unplugged is fundamentally difficult:

- **Limited coverage.** The action space is very large, the state-action coverage in the dataset is low, and the environment is highly partially observable. This makes it challenging for the value function to extrapolate to unseen state and actions (Fujimoto et al., 2019; Gulcehre et al., 2022). This is particularly impactful for Q-values based methods, since there are significantly fewer states than state-action pairs, so it is easier to learn state value functions. The approaches like R-BVE mitigate extrapolation errors during training, but still the agent has to extrapolate during inference.
- **Weak learning signal.** The win-loss reward is a weak learning signal to learn a good policy because the dataset has games with a wide range of qualities and the win-loss reward ignores this. For example, the winner of a game between two low-skilled players would consistently lose to the loser of a game between two professional players. Thus, purely relying on the win-loss signal in the offline RL case is problematic.
- **Credit assignment** is difficult due to large action space, sparse rewards, long-horizon, and partial observability. This exacerbates the problems with the offline RL based agents.
- **Autoregressive action space** requires learning autoregressive Q-values which is challenging and understudied in the literature. In this paper, we side-stepped this by just learning a Q-function only for the `function` argument.

7. Related work

Online RL has been very impactful for building agents to play computer games. RL agents can outperform professional human players in many games such as StarCraft II (Vinyals et al., 2019), DOTA (Berner et al., 2019) or Atari (Badia et al., 2020; Mnih et al., 2015). Similar levels of progression have been observed on board games, including chess and Go (Silver et al., 2016, 2017). Although offline RL approaches have shown promising results on Atari recently (Schrittwieser et al., 2021b), they have not been previously applied on complex partially observable games using data derived from human experts.

RL Unplugged (Gulcehre et al., 2020) introduces a suite of benchmarks for Offline RL with a diverse set of task domains with a unified API and evaluation protocol. D4RL (Fu et al., 2020) is an offline RL benchmark suite focusing only on mixed data sources. However, both RL Unplugged and D4RL lack high-dimensional, partially observable tasks. This paper fills that gap by introducing a benchmark for StarCraft II.

Offline RL has become an active research area, as it enables us to leverage fixed datasets to learn policies to deploy in the real-world. Offline RL methods include 1) policy-constraint approaches that regularize the learned policy to stay close to the behavior policy (Fujimoto et al., 2019; Wang et al., 2020), 2) value-based approaches that encourage more conservative value estimates, either through a pessimistic regularization or uncertainty (Gulcehre et al., 2021; Kumar et al., 2020), 3) model-based approaches (Kidambi et al., 2020; Schrittwieser et al., 2021b; Yu et al., 2020), and 4) adaptations of standard off-policy RL methods such as DQN (Agarwal et al., 2020) or D4PG (Wang et al., 2020). Recently methods using only one-step of policy improvement has been proven to be very effective on offline reinforcement learning (Brandfonbrener et al., 2021; Gulcehre et al., 2021).

8. Conclusions

Offline RL has enabled the deployment of RL ideas to the real world. Academic interest in this area has grown and several benchmarks have been proposed, including RL-Unplugged (Gulcehre et al., 2020), D4RL (Fu et al., 2020), and RWRL (Dulac-Arnold et al., 2019). However, because of the relatively small-scale and synthetic nature of these benchmarks, they don't capture the challenges of real-world offline RL.

In this paper, we introduced AlphaStar Unplugged, a benchmark to evaluate agents which play StarCraft II by learning only from *offline* data. This data is comprised of over a million games games mostly played by amateur human StarCraft II players on Blizzard's Battle.Net.[11] Thus, the benchmark more accurately captures the challenges of offline RL where an agent must learn from logged data, generated by a diverse group of weak experts, and where the data doesn't exhaust the full state and action space of the environment.

We showed that offline RL algorithms can exceed 90% win rate against the all-races version of the previously published AlphaStar Supervised agent (trained using behavior cloning). However, the gap between online and offline methods still exists and we hope the benchmark will serve as a testbed to advance the state of art in offline RL algorithms.

Acknowledgments

We would like to thank Alistair Muldal for helping with several aspects of the open-sourcing which went a long way in making the repository user-friendly. We would like to thank Scott Reed and

[11] https://en.wikipedia.org/wiki/Battle.net

David Silver for reviewing the manuscript, the AlphaStar team (Vinyals et al., 2019) for sharing their knowledge and experience about the game. We would like to thank the authors of MuZero Unplugged (Schrittwieser et al., 2021a) and Sampled MuZero (Hubert et al., 2021) for advising on the development of the MuZero Supervised agent. We also thank the wider DeepMind research, engineering, and environment teams for the technical and intellectual infrastructure upon which this work is built. We are grateful to the developers of tools and frameworks such as JAX (Babuschkin et al., 2020), Haiku (Hennigan et al., 2020) and Acme (Hoffman et al., 2020) that enabled this research.

References

Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning*, pp. 104–114. PMLR, 2020.

OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Claudio Fantacci, Jonathan Godwin, Chris Jones, Tom Hennigan, Matteo Hessel, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Lena Martens, Vladimir Mikulik, Tamara Norman, John Quan, George Papamakarios, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Wojciech Stokowiec, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL http://github.com/deepmind.

Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Zhaohan Daniel Guo, and Charles Blundell. Agent57: Outperforming the atari human benchmark. In *International Conference on Machine Learning*, pp. 507–517. PMLR, 2020.

Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 1957.

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

David Brandfonbrener, William F Whitney, Rajesh Ranganath, and Joan Bruna. Offline rl without off-policy evaluation. *arXiv preprint arXiv:2106.08909*, 2021.

David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. When does return-conditioned supervised learning work for offline reinforcement learning? *arXiv preprint arXiv:2206.01079*, 2022.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc Viet Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In Anna Korhonen, David R. Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*,

pp. 2978–2988. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1285. URL https://doi.org/10.18653/v1/p19-1285.

Trevor Davis, Neil Burch, and Michael Bowling. Using response functions to measure strategy strength. In *Twenty-Eighth AAAI Conference on Artificial Intelligence*, 2014.

Gabriel Dulac-Arnold, Daniel Mankowitz, and Todd Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.

Arpad E Elo. *The rating of chessplayers, past and present*. Arco Pub., 1978.

Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. Rvs: What is essential for offline rl via supervised learning? *arXiv preprint arXiv:2112.10751*, 2021.

Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: scalable distributed deep-rl with importance weighted actor-learner architectures. *CoRR*, abs/1802.01561, 2018. URL http://arxiv.org/abs/1802.01561.

Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, pp. 2052–2062. PMLR, 2019.

Caglar Gulcehre, Ziyu Wang, Alexander Novikov, Tom Le Paine, Sergio Gomez Colmenarejo, Konrad Zolna, Rishabh Agarwal, Josh Merel, Daniel Mankowitz, Cosmin Paduraru, et al. Rl unplugged: Benchmarks for offline reinforcement learning. *arXiv e-prints*, pp. arXiv–2006, 2020.

Caglar Gulcehre, Sergio Gómez Colmenarejo, Ziyu Wang, Jakub Sygnowski, Thomas Paine, Konrad Zolna, Yutian Chen, Matthew Hoffman, Razvan Pascanu, and Nando de Freitas. Regularized behavior value estimation. *arXiv preprint arXiv:2103.09575*, 2021.

Caglar Gulcehre, Srivatsan Srinivasan, Jakub Sygnowski, Georg Ostrovski, Mehrdad Farajtabar, Matt Hoffman, Razvan Pascanu, and Arnaud Doucet. An empirical study of implicit regularization in deep offline rl. *arXiv preprint arXiv:2207.02099*, 2022.

Lei Han, Jiechao Xiong, Peng Sun, Xinghai Sun, Meng Fang, Qingwei Guo, Qiaobo Chen, Tengfei Shi, Hongsheng Yu, Xipeng Wu, and Zhengyou Zhang. Tstarbot-x: An open-sourced and comprehensive study for efficient league training in starcraft ii full game, 2021.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100. AAAI Press, 2016.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

Tom Hennigan, Trevor Cai, Tamara Norman, and Igor Babuschkin. Haiku: Sonnet for JAX, 2020. URL http://github.com/deepmind/dm-haiku.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, nov 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, Sarah Henderson, Alex Novikov, Sergio Gómez Colmenarejo, Serkan Cabi, Caglar Gulcehre, Tom Le Paine, Andrew Cowie, Ziyu Wang, Bilal Piot, and Nando de Freitas. Acme: A research framework for distributed reinforcement learning. *arXiv preprint arXiv:2006.00979*, 2020. URL https://arxiv.org/abs/2006.00979.

Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Mohammadamin Barekatain, Simon Schmitt, and David Silver. Learning and planning in complex action spaces. *arXiv preprint arXiv:2104.06303*, 2021.

Ray Jiang, Tom Zahavy, Adam White, Zhongwen Xu, Matteo Hessel, Charles Blundell, and Hado van Hasselt. Emphatic algorithms for deep reinforcement learning. In *International Conference on Machine Learning*. PMLR, 2021.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. *arXiv preprint arXiv:2005.05951*, 2020.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *arXiv preprint arXiv:2006.04779*, 2020.

Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *CoRR*, abs/1910.00177, 2019.

G. Rummery and Mahesan Niranjan. On-line q-learning using connectionist systems. *Technical Report CUED/F-INFENG/TR 166*, 11 1994.

Mikayel Samvelyan, Tabish Rashid, Christian Schröder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob N. Foerster, and Shimon Whiteson. The starcraft multi-agent challenge. *CoRR*, abs/1902.04043, 2019. URL http://arxiv.org/abs/1902.04043.

Julian Schrittwieser, Thomas Hubert, Amol Mandhane, Mohammadamin Barekatain, Ioannis Antonoglou, and David Silver. Online and offline reinforcement learning by planning with a learned model. *arXiv preprint arXiv:2104.06294*, 2021a.

Julian Schrittwieser, Thomas Hubert, Amol Mandhane, Mohammadamin Barekatain, Ioannis Antonoglou, and David Silver. Online and offline reinforcement learning by planning with a learned model. *arXiv preprint arXiv:2104.06294*, 2021b.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Rupesh Kumar Srivastava, Pranav Shyam, Filipe Mutz, Wojciech Jaśkowski, and Jürgen Schmidhuber. Training agents using upside-down reinforcement learning. *arXiv preprint arXiv:1912.02877*, 2019.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2018.

John N. Tsitsiklis and B. Van Roy. An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5):674–690, 1997.

Hado van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. *CoRR*, abs/1812.02648, 2018. URL http://arxiv.org/abs/1812.02648.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017a.

Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks, 2017b.

Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pp. 1995–2003. PMLR, 2016.

Ziyu Wang, Alexander Novikov, Konrad Zolna, Jost Tobias Springenberg, Scott Reed, Bobak Shahriari, Noah Siegel, Josh Merel, Caglar Gulcehre, Nicolas Heess, et al. Critic regularized regression. *arXiv preprint arXiv:2006.15134*, 2020.

Ian H. Witten. An adaptive optimal controller for discrete-time markov environments. *Information and Control*, 34:286–295, 1977.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. *arXiv preprint arXiv:2005.13239*, 2020.

A. Appendix

A.1. StarCraft II Interface

StarCraft II features large maps on which players move their units. They can also construct buildings (units which cannot move), and gather resources. At any given time, they can only observe a subset of the whole map through the *camera*, as well as a coarse, zoomed-out version of the whole map called the *minimap*. In addition, units have a *vision* field such that any unit owned by the opponent is hidden unless it is in a vision field. Human players play through the *standard interface*, and receive some additional information, such as the quantity of resources owned or some details about the units currently selected, and audible cues about events in the game. Players issue orders by first selecting units, then choosing an ability, and lastly, for some actions, a target, which can be on the camera or the minimap.

While human players use their mouse and keyboard to play the game, the agents use an API, called the *raw interface*, which differs from the standard interface.[12] In this interface, observations are split into three modalities, namely `world`, `units` and `vectors` which are enough to describe the full observation:

- **World.** A single tensor called *world* which corresponds to the minimap shown to human players. It is observed as 8 feature maps with resolution 128x128. The feature maps are:
  - `height_map`: The topography of the map, which stays unchanged throughout the game.
  - `visibility_map`: The area within the vision of any of the agent's units.
  - `creep`: The area of the map covered by Zerg "creep".
  - `player_relative`: For each pixel, if a unit is present on this pixel, this indicates whether the unit is owned by the player, the opponent, or is a neutral construct.
  - `alerts`: This encodes alerts shown on the minimap of the standard interface, for instance when units take damage.
  - `pathable`: The areas of the map which can be used by ground units.
  - `buildable`: The areas of the map which where buildings can be placed.
  - `virtual_camera`: The area currently covered by the virtual camera. The virtual camera restricts detailed vision of many `unit` properties, as well as restricts some actions from targeting units/points outside the camera. It can be moved as an action.
- **Units.** A list of units observed by the agent. It contains all of the agent's units as well as the opponent's units when within the agent's vision and the last known state of opponent's buildings. For each unit, the observation is a vector of size 43 containing all the information that would be available in the game standard interface. In particular, some of the opponent's unit information are masked if they are not in the agent's virtual camera. In addition, the list also contains entries for *effects* which are temporary, localized events in the game (although effects are not units in a strict sense, they can be represented as such). In our implementation, this list can contain up to 512 entities. In the rare event that more than 512 units/effects exist at once, the additional observations will be truncated and not visible to the agent.
- **Vectors.** Global inputs are gathered into `vectors`. They are:
  - `player_id`: The id of the player (0 or 1). This is not useful to the agent.
  - `minerals`: The amount of minerals currently owned.
  - `vespene`: The amount of vespene gas currently owned.

[12] Although the raw interface is designed to be as fair as possible when compared to the standard interface, in particular the observation and actions do not contain significantly different information between the two.

- `food_used`: The current amount of food currently used by the agent's units. Different units use different amount of food and players need to build structures to raise the `food_cap`.
- `food_cap`: The current amount of food currently available to the agent.
- `food_used_by_workers`: The current amount of food currently used by the agent's workers. Workers are basic units which harvest resources and build structures, but rarely fight.
- `food_used_by_army`: The current amount of food currently used by the agent's non-worker units.
- `idle_worker_count`: The number of workers which are idle. Players typically want to keep this number low.
- `army_count`: The number of units owned by the agent which are not workers.
- `warp_gate_count`: The number of warp gates owned by the agent (if the agent's race is Protoss).
- `larva_count`: The number of larva currently available to the agent (if the agent's race is Zerg).
- `game_loop`: The number of internal game steps since the beginning of the game.
- `upgrades`: The list of upgrades currently unlocked by the agent.
- `unit_counts`: The number of each unit currently owned by the agent. This information is contained in the units input, but represented in a different way here.
- `home_race`: The race of the agent.
- `away_race`: The race of the opponent. If the opponent is has chosen a random race, it is hidden until one of their unit is observed for the first time.
- `prev_delay`: The number of internal game steps since the last observation. During inference, this can be different from the `delay` argument of the previous action (see Section 3.1), for instance if there was lag.

Each action from the raw interface combines up to three standard actions: unit selection, ability selection and target selection. In practice, each raw action is subdivided into up to 7 parts, called *arguments*. It is important to note that the arguments are not independent of each other, in particular the `function` determines which other arguments are used. The arguments are detailed below:

- **Function.** This corresponds to the ability part of the StarCraft II API, and specifies the action. Examples include: `Repair`, `Train_SCV`, `Build_CommandCenter` or `Move_Camera`.
- **Delay.** In theory, the agent could take an action at each environment step. However, since StarCraft II is a real-time game, the internal game steps are very quick[13] and therefore it would not be fair to humans, which cannot issue action that fast. In addition, it would make episodes extremely long which is challenging to learn. Therefore the agent specifies how many environment steps will occur before the next observation-action pair. Throttling is used to make sure the agent cannot issue too many actions per second.
- **Queued.** This argument specifies whether this action should be applied immediately, or queued. This corresponds to pressing the Shift key in the standard game interface.
- **Repeat.** Some repeated identical actions can be issued very quickly in the standard interface, by pressing keyboard keys very fast, sometimes even issuing more than one action per internal game step. The repeat argument lets the agent repeat some actions up to 4 times in the same step. This is mainly useful for building lots of Zerg units quickly.
- **Unit tags.** This is the equivalent of a selection action in the standard interface. This argument is a mask over the agent's units which determines which units are performing the action. For instance for a `Repair` action, the unit tags argument specify which units are going to perform

[13] 22.4 steps per second.

the repair.
- **Target unit tag.** Which unit an action should target. For instance, a `Repair` action needs a specific unit/building to repair. Some actions (e.g. `Move`) can target either a unit or a point. Those actions are split into two functions. There are no actions in StarCraft II that target more than one unit (some actions can affect more than one unit, but those actions specify a target point).
- **World.** Which point in the world this action should target. It is a pair of (x, y) coordinates aligned with the world observation. For example `Move_Camera` needs to know where to move the camera.

A.2. Evaluation Metrics

Let us assume that we are given an agent to evaluate p and a collection of reference agents

$$\mathrm{Q} = \{\mathrm{q}_j\}_{j=1}^{N}.$$

Each of these players can play all three races of StarCraft II:

$$\mathrm{R} = \{\text{terran}, \text{protoss}, \text{zerg}\}.$$

We define an outcome of a game between a player p and a reference player q as

$$\mathrm{f}(\mathrm{p}, \mathrm{q}) \stackrel{\text{def}}{=} \mathbb{E}_{\mathrm{r_p}, \mathrm{r_q} \sim U(\mathrm{R})} \mathrm{P}[\mathrm{p \text{ wins against } q} | \mathrm{r}(\mathrm{p}) = \mathrm{r_p}, \mathrm{r}(\mathrm{q}) = \mathrm{r_q}],$$

where $\mathrm{r}(\cdot)$ returns a race assigned to a given player, and the probability of winning is estimated by playing matches over uniformly samples maps and starting locations.

A.2.1. Robustness computation

We define robustness of an agent p with respect to reference agents Q as

$$\mathrm{robustness}_{\mathrm{Q}}(\mathrm{p}) \stackrel{\text{def}}{=} 1 - \min_{\mathrm{q} \in \mathrm{Q}} \mathrm{f}(\mathrm{p}, \mathrm{q}).$$

Note, this is 1 minus exploitability Davis et al. (2014), simply flipped so that we maximise the score. In particular, Nash equlibrium would maximise this metric, if Q contained every mixed strategy in the game.

A.2.2. Elo computation

We follow a standard Chess Elo model Elo (1978), that tries to predict f by associating each of the agents with a single scalar (called Elo rating) $\mathrm{e}(\cdot) \in \mathbb{R}$ and then modeling outcome with a logistic model:

$$\widehat{\mathrm{f_{Elo}}}(\mathrm{p}, \mathrm{q}) \stackrel{\text{def}}{=} \frac{1}{1 + 10^{[\mathrm{e}(\mathrm{p}) - \mathrm{e}(\mathrm{q})]/400}}.$$

For consistency of the evaluation we have precomputed ratings $\mathrm{e}(\mathrm{q})$ for each $\mathrm{q} \in \mathrm{Q}$. Since ratings are invariant to translation, we anchor them by assigning $\mathrm{e}(\text{very_hard}) := 1000$.

In order to compute rating of a newly evaluated agent p we minimise the cross entropy between true, observed outcomes f and predicted ones (without affecting the Elo ratings of reference agents):

$$\mathrm{Elo}_{\mathrm{Q}}(\mathrm{p}) \stackrel{\text{def}}{=} \arg\min_{\mathrm{e}(\mathrm{p})} \left[- \sum_{\mathrm{q}} \mathrm{f}(\mathrm{p}, \mathrm{q}) \log \left(\widehat{\mathrm{f_{Elo}}}(\mathrm{p}, \mathrm{q}) \right) \right] = \arg\max_{\mathrm{e}(\mathrm{p})} \sum_{\mathrm{q}} \mathrm{f}(\mathrm{p}, \mathrm{q}) \log \left(\widehat{\mathrm{f_{Elo}}}(\mathrm{p}, \mathrm{q}) \right).$$

Note, that as it is a logistic model, it will be ill defined if f(p, q) = 1 (or 0) for all q (one could say Elo is infinite). In such situation the metric will be saturated, and one will need to expand Q to continue research progress.

A.3. Training of reference agents

In this section, we present the details of the training of the reference agents. We list the hyperparameters and propose pseudocode implementations for each agent.

In the pseudo-code, the data at a time step t is denoted as X. We refer to different part of the data X: X_s is the observation at time t, X_a is the action at time t, X_r is the reward at time t, X_R is the MC return (in the case of StarCraft II, this is equal to the reward on the last step of the episode), and $X_{\text{game loop delta}}$ is the number of internal game steps between t and $t+1$ (and 0 on the first step).

A.3.1. Behavior Cloning (BC)

This agent is trained to minimize the behavior cloning loss L^{BC} (see Section 4.3), with weight decay. We used a cosine learning rate schedule with $\lambda_0 = 5 \cdot 10^{-4}$ and no ramp-in, over a total of $k_{max} = 10^{10}$ training frames. We used a rollout length $K = 1$ and a minibatch size $M = 32,768$. We used the Adam optimizer (Loshchilov & Hutter, 2019), and we clip the gradients to 10 before applying Adam.

Algorithm 1 Behavior Cloning (with rollout length K set to 1)

Inputs: A dataset of trajectories $\mathcal{D}$, a mini batch size M, an initial learning rate λ, the total number of observations processed n_{frames}, and the initial weights used θ to parameterise the estimated policy $\hat{\mu}_\theta$.
for $i = 0..n_{\text{frames}}/M - 1$ **do**
  Set the gradient accumulator $g_{\text{acc}} \leftarrow 0$.
  for $j = 0..M - 1$ **do**
    Sample a trajectory $T \sim \mathcal{D}$
    Sample k in $[0, length(T) - K]$
    Set $X \leftarrow T[k]$
    Set $g_{\text{acc}} \leftarrow g_{\text{acc}} + \frac{1}{M} \cdot \left(\frac{\partial L_{\text{cross entropy}}(\hat{\mu}_\theta(\cdot|X_s), X_a)}{\partial \theta} \right)$,
    where X_s, X_a are the observation and action parts of X, respectively.
  end for
  Set $\lambda_i \leftarrow \frac{\lambda}{2} \left(\cos\left(\frac{i\pi}{n_{\text{frames}}/M - 1} \right) + 1 \right)$
  Set $\theta \leftarrow \theta - \lambda_i \cdot Adam(g_{\text{acc}})$
end for
return θ

A.3.2. Fine-Tuned Behavior Cloning (FT-BC)

For the FT-BC agent, we initialized the weights using the trained BC agent. We then trained it to minimize L^{BC} on $k_{max} = 10^9$ frames, with a cosine learning rate schedule with $\lambda_0 = 10^{-5}$ and $N_{\text{ramp-in}} = 10^8$. We used a rollout length $K = 1$ and a minibatch size $M = 32,768$. The data was restricted to episodes with $MMR > 6200$ and reward $r = 1$. We used the Adam optimizer, and we clip the gradients to 10 before applying Adam.

A.3.3. Offline Actor-Critic (OAC)

For the OAC agent, we first trained a behavior cloning agent with a value function by minimizing

$$10 \cdot L^{MSE} + L^{BC} \tag{9}$$

with the same parameters are the BC agent training, except that weight decay was disabled.

From this model, we then trained using the offline actor-critic loss, minimizing L^{VTrace} for $K_{max} = 10^9$ frames, using a rollout length $K = 64$ and a minibatch size $M = 512$. We found that it is important to clip the gradients to 10 after applying Adam during this phase. We used a per-internal game step discount $\gamma = 0.99995$.

The loss L^{VTrace} is the policy loss defined in Espeholt et al. (2018), and a a bit more complex than $L^{TD(0)}$ presented in Section 4.4. We used mixed n-steps TD, with n between 32 and 64. As part of the V-Trace computation, we clipped ρ and c to 1 (see Espeholt et al. (2018)).

Value function reaches 72% accuracy, which is computed as the fraction of steps where the sign of V^{μ} is the same as R.

Divergence: We observe that when doing so, using the value function V^{π} leads to divergence during training, as shown on Figure 7 in the Appendix.

Algorithm 2 Behavior Cloning with value function training (with rollout length K set to 1)

Inputs: A dataset of trajectories $\mathcal{D}$, a mini batch size M, an initial learning rate λ, the total number of observations processed n_{frames}, and the initial weights used θ to parameterise the estimated policy $\hat{\mu}_\theta$ and value function $V^{\hat{\mu}_\theta}$.
for $i = 0..n_{\text{frames}}/M - 1$ **do**
  Set the gradient accumulator $g_{\text{acc}} \leftarrow 0$.
  for $j = 0..M - 1$ **do**
    Sample a trajectory $T \sim \mathcal{D}$
    Sample k in $[0, length(T) - K]$
    Set $X \leftarrow T[k]$
    Set $g_{\text{acc}} \leftarrow g_{\text{acc}} + \frac{1}{M} \cdot \left(\frac{\partial L_{\text{cross entropy}}(\hat{\mu}_\theta(\cdot|X_s), X_a)}{\partial \theta} + \frac{\partial L_{\text{MSE}}(V^{\hat{\mu}}(X_s), X_r)}{\partial \theta} \right)$,
    where X_s, X_a are the observation and action parts of X, respectively.
  end for
  Set $\lambda_i \leftarrow \frac{\lambda}{2} \left(\cos\left(\frac{i\pi}{n_{\text{frames}}/M - 1} \right) + 1 \right)$
  Set $\theta \leftarrow \theta - \lambda_i \cdot Adam(g_{\text{acc}})$.
end for
return θ

A.3.4. Emphatic Offline Actor-Critic (E-OAC)

Because of the way emphatic traces are computed, the E-OAC agent requires learning from consecutive minibatches[14]. Details can be found in Appendices A.3.3 and A.3.4. As explained in Appendix A.1, we only apply policy improvement to the `function` and `delay` arguments of the action for simplicity.

The E-OAC agent uses the same BC agent as the OAC training in the previous section, that is, θ_0 is also set to θ_V. Then we run Algorithm 4 with the same hyper-parameters as the OAC agent. However,

[14] Such that the first element of each rollout of a minibatch are adjacent to the last element of each rollouts of the previous minibatch

Algorithm 3 Offline Actor-Critic (with fixed critic V^{μ})

Inputs: A dataset of trajectories $\mathcal{D}$, the mini batch size M, the rollout length K, an initial learning rate λ, the weights of the estimated behavior policy and value function from BC θ_0 (such that $\hat{\mu}_{\theta_0}$ is the BC policy, and $V^{\hat{\mu}_{\theta_0}}$ is the behavior value function), the total number of observations processed n_{frames}, the bootstrap length N, the IS threshold $\bar{\rho}$, a per-game step discount γ_0.
Set $\theta \leftarrow \theta_0$
for $i = 0..n_{\text{frames}}/M - 1$ **do**
  Set the gradient accumulator $g_{\text{acc}} \leftarrow 0$.
  for $j = 0..M - 1$ **do**
    Sample a trajectory $T \sim \mathcal{D}$
    Sample k in $[0, length(T) - K]$
    Set $X \leftarrow T[k : k + K - 1]$
    Compute the TD errors δ and clipped IS ratios $\bar{\rho}$ with clipping threshold $\hat{\rho}$
    for $t = 0..K - 2$ **do**
      Set $\delta[t] \leftarrow X_r[t+1] + \gamma_{t+1} V^{\hat{\mu}_{\theta_0}}(X_s[t+1]) - V^{\hat{\mu}_{\theta_0}}(X_s[t])$
      Set $\bar{\rho}[t] \leftarrow \min(\bar{\rho}, \frac{\pi_\theta(X_a[t]|X_s[t])}{\hat{\mu}_{\theta_0}(X_a[t]|X_s[t])})$
      Set $\gamma[t] \leftarrow \gamma_0^p$ where $p = X_{\text{game_loop_delta}}[t]$.
    end for
    Compute the V-Trace targets v:
    for $t = 0..K - N - 1$ **do**
      Set $v[t+1] \leftarrow V^{\hat{\mu}_{\theta_0}}(X_s[t+1]) + \sum_{u=t}^{t+N-1} (\prod_{k=t}^{u-1} \bar{\rho}[k]\gamma[k+1])\ \bar{\rho}[u]\delta[u]$
    end for
    Set $g_{\text{acc}} \leftarrow g_{\text{acc}} + \sum_{t=0}^{N-1} \bar{\rho}[t](X_r[t+1] + \gamma[t+1]v[t+1] - V^{\hat{\mu}_{\theta_0}}(X_s[t])) \frac{\partial \log \pi_\theta(X_a[t]|X_s[t])}{\partial \theta}$.
  end for
  Set $\lambda_i \leftarrow \frac{\lambda}{2}\left(\cos\left(\frac{i\pi}{n_{\text{frames}}/M-1}\right) + 1\right)$
  Set $\theta \leftarrow \theta - \lambda_i \cdot Adam(g_{\text{acc}})$
end for
return θ

unlike the OAC agent, it uses sequentially ordered trajectories in their order of interactions with the MDP, and reweight the policy gradient updates with the emphatic trace F.

Algorithm 4 Emphatic Offline Actor-Critic (with fixed critic V^{μ})

Inputs: A dataset of trajectories $\mathcal{D}$, the mini batch size M, the rollout length K, an initial learning rate λ, the weights of the estimated behavior policy and value function from BC θ_0 (such that $\hat{\mu}_{\theta_0}$ is the BC policy, and $V^{\hat{\mu}_{\theta_0}}$ is the behavior value function), the total number of observations processed n_{frames}, the bootstrap length N, the IS threshold $\bar{\rho}$, a buffer $\mathcal{B}$ containing M empty lists, a per-game step discount γ_0, initial emphatic traces $\forall j < N, F[j] = 1$.
Set $\theta \leftarrow \theta_0$
for $i = 0..n_{\text{frames}}/M - 1$ **do**
  Set the gradient accumulator $g_{\text{acc}} \leftarrow 0$.
  for $j = 0..M - 1$ **do**
    if $\mathcal{B}[j]$ has less than $K + 1$ elements **then**
      Sample $T \sim \mathcal{D}$
      $\mathcal{B}[j] \leftarrow \text{concatenate}(\mathcal{B}[j], T)$
    end if
    Set $X \leftarrow \mathcal{B}[j][0 : K + 1]$
    Set $\mathcal{B}[j] \leftarrow \mathcal{B}[j][K :]$
    Compute the TD errors δ, clipped IS ratios $\bar{\rho}$ and V-Trace targets v with clipping threshold $\hat{\rho}$ as in Alg. 3.
    Compute the emphatic trace F:
    for $t = 0..K - N - 1$ **do**
      Set $F[t] = \prod_{p=1}^{N}(\gamma[t - p + 1]\hat{\rho}[t - p])F[t - N] + 1$
    end for
    Set $g_t = \bar{\rho}[t](X_r[t + 1] + \gamma[t + 1]v[t + 1] - V^{\hat{\mu}_{\theta_0}}(X_s[t]))\frac{\partial \log \pi_\theta(X_a[t]|X_s[t])}{\partial \theta}$
    Set $g_{\text{acc}} \leftarrow g_{\text{acc}} + \sum_{t=0}^{N-1} F[t]g_t$
  end for
  Set $\lambda_i \leftarrow \frac{\lambda}{2}\left(\cos\left(\frac{i\pi}{n_{\text{frames}}/M-1}\right) + 1\right)$
  Set $\theta \leftarrow \theta - \lambda_i \cdot Adam(g_{\text{acc}})$
end for
return θ

A.3.5. MuZero (MZS and MZS-MCTS)

We used AlphaStar's encoders and action prediction functions in the MuZero architecture. AlphaStar's action prediction functions are notably autoregressive to handle the complex and combinatorial action space of StarCraft II. We predict and embed the full action in the representation function, *i.e.* for the root node in MCTS. To improve inference time, we only predict the `function` and `delay` arguments in the prediction heads in the model, *i.e.* for non-root nodes in MCTS. We found that using as few as 20 actions for Sampled MuZero worked well, and increasing it did not improve performance.

We use a target network which is updated every 100 learner steps, to compute the bootstrapped target value v^{-}_{t+n}. We found that $n = 512$ worked best, which is notably large and roughly corresponds to half of the average game length.

Note that here we differ from MuZero and MuZero Unplugged by training with action and value targets obtained from the offline dataset. MuZero and MuZero Unplugged use the result of MCTS as the action and value targets.

We use Adam optimizer with additive weight decay (Loshchilov & Hutter, 2017), and a cosine learning rate schedule with $\lambda_0 = 10^{-3}$.

Algorithm 5 MuZero Supervised

Inputs: A dataset of trajectories $\mathcal{D}$, the mini batch size M, the rollout length K, the temporal difference target distance n, an initial learning rate λ, the total number of observations processed n_{frames}, and the initial weights used θ to parameterise the representation function h_θ, the dynamics function g_θ, and the prediction function f_θ.
for $i = 0..n_{\text{frames}}/M - 1$ **do**
  Set the gradient accumulator $\nabla_{\text{acc}} \leftarrow 0$.
  for $j = 0..M - 1$ **do**
    Sample a trajectory $T \sim \mathcal{D}$
    Sample k in $[0, length(T))$
    Set $X \leftarrow T[k : k + K]$
    Set $X_{\text{TD}} \leftarrow T[k + n]$
    Set $\nabla_{\text{acc}} \leftarrow \nabla_{\text{acc}} + \frac{1}{M} \cdot \frac{\partial \mathcal{L}_{\text{MuZero}}(\theta, X_s[k], X_a[k:k+K], X_{\text{TD}})}{\partial \theta}$.
  end for
  Set $\lambda_i \leftarrow \frac{\lambda}{2}\left(\cos\left(\frac{i\pi}{n_{\text{frames}}/M-1}\right) + 1\right)$
  Set $\theta \leftarrow \theta - \lambda_i \cdot Adam(g_{\text{acc}})$.
end for
return θ

A.4. Expanded win rate matrix

In this section, we present an expanded version of the win rate matrix of our reference agents shown in Figure 9. See Section 5.11 for more details.

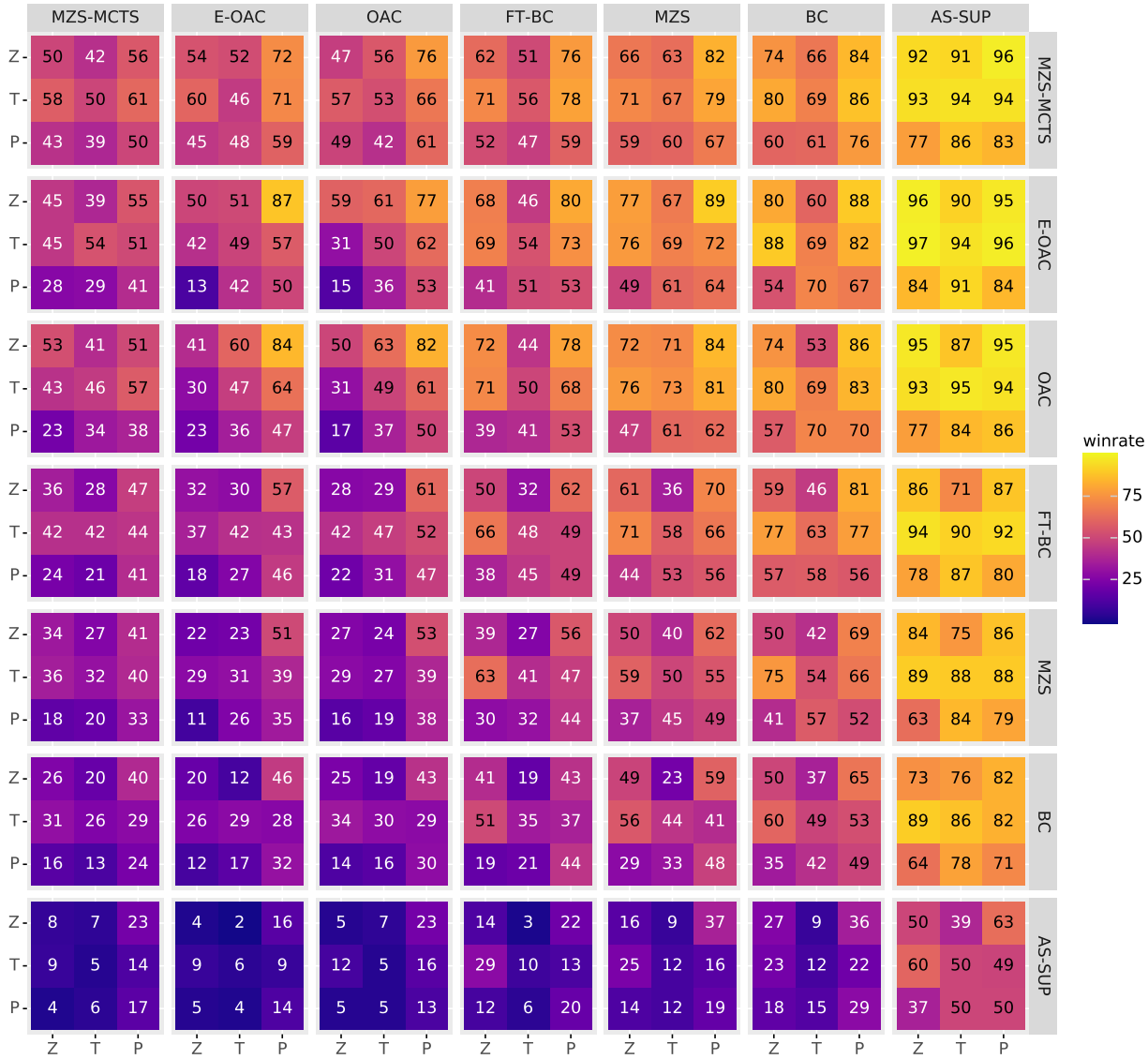


Figure 10 | Win rate matrix of the reference agents broken down by race (Protoss, Terran, Zerg), normalized between 0 and 100. Note that because of draws, the win rates do not always sum to 100 across the diagonal. AS-SUP corresponds the the original AlphaStar Supervised agent trained to play all races.